

Hướng 2 — GNN-RAG-inspired Conflict Zone

Điểm khác biệt cốt lõi so với Hướng 1

- **Hướng 1** dùng arbitration mechanism (giống ArbGraph) để tính credibility score — propagate signals qua graph theo rule-based iterative update.
- **Hướng 2** thay thế bằng GNN được train để học conflict-aware reasoning trực tiếp trên evidence graph — GNN đóng vai trò "dense subgraph reasoner" giống GNN-RAG, nhưng thay vì score answer nodes thì score conflict intensity và claim reliability của từng node.

Pipeline (các phase 1-4 giữ nguyên, thay đổi từ phase 5):

documents → claim extraction → claim canonical → evidence graph → retrieve → [THAY ĐỔI] GNN-based Conflict Zone → [THAY ĐỔI] Path-guided Iterative Loop → generation

Phase 5 — GNN-based Conflict Zone

5.1 — GNN Reasoning trên Evidence Graph

Thay vì dùng rule-based arbitration để propagate credibility, train một GNN để học cách score từng claim node dựa trên cấu trúc graph xung quanh nó.

GNN nhận evidence graph $G = (V, E)$ và query embedding q , thực hiện message passing:

Plaintext

$$h(l)_v = \psi(h(l-1)_v, \sum \omega(q, e_{vv'}) \cdot m(l)_{vv'})$$

trong đó:

$\omega(q, e_{vv'})$ = relevance của edge $e_{vv'}$ với query q

$m(l)_{vv'}$ = message từ neighbor v' sang v

ψ = combine function (MLP)

Sau L layers, mỗi node v có final representation $h(L)_v$. GNN được train để output hai scores per node:

Plaintext

$$\text{claim_reliability_score}(v) = \text{sigmoid}(W_r \cdot h(L)_v)$$

$$\text{conflict_node_score}(v) = \text{sigmoid}(W_c \cdot h(L)_v)$$

Training objective: supervised trên datasets có ground truth conflict labels (ConflictBank, ECON)
 — binary cross-entropy cho reliability, MSE cho conflict intensity.

5.2 — Factoid Decomposition (giữ nguyên từ Hướng 1)

Với mỗi high conflict_node_score pair (v_i, v_j):

Plaintext

$\text{claim}_i \rightarrow S_i = \{\text{temporal, numerical, entity_subject, entity_object, relation, location}\}$

$\text{claim}_j \rightarrow S_j = \{\text{temporal, numerical, entity_subject, entity_object, relation, location}\}$

5.3 — Conflict Subgraph Extraction (mới — từ GNN-RAG)

Thay vì chỉ lưu conflict pairs, hệ thống extract shortest reasoning paths giữa conflict nodes — giống GNN-RAG extract shortest paths từ question entities đến answer candidates.

Với conflict pair (v_i, v_j):

Plaintext

$\text{path}_{ij} = \text{shortest_path}(v_i, v_j, G) = v_i \rightarrow r_k \rightarrow r_l \rightarrow v_j$

Mỗi edge trên path được verbalize:

"r1 [support] r_k [contradict] v_j"

Reasoning paths này capture chuỗi logic dẫn đến conflict — không chỉ biết hai claims mâu thuẫn mà còn biết tại sao thông qua intermediate claims.

5.4 — Conflict Localization (giữ nguyên + bổ sung path context)

Factoid alignment $\rightarrow$ Binary vectors p_i, p_j

JSON

```
conflict_localization = {
  "slot": "temporal",
  "values": ["1954", "1953"],
  "intensity": 0.33,
  "reasoning_path": "r1 [support] r6 [contradict] r3",
  "path_verbalized": "Source A support by r6, r6 contradict Source B"
}
```

Output tại phase:

- Danh sách conflict nodes + `claim_reliability_score` (từ GNN)
- `conflict_intensity_score` (từ factoid XOR)
- `reasoning_path` (từ shortest path extraction)

Phase 6 — Path-guided Iterative Loop

Điểm khác biệt so với Hướng 1

- **Hướng 1:** inject conflict factoids vào query → retrieve blindly.
- **Hướng 2:** dùng reasoning path như retrieval guidance — mỗi node trên path là một "stepping stone" để formulate targeted sub-queries, giống multi-hop retrieval của GNN-RAG.

6.1 — Path-guided Query Decomposition

- **Conflict pair:** (r1: "1954", r3: "1953")
- **Reasoning path:** r1 → r6 → r3

Decompose thành sub-queries theo path:

- **Sub-query 1:** "Verify r6: [intermediate claim]"
- **Sub-query 2:** "Verify connection r6 → r3"
- **Sub-query 3:** "Find additional evidence for temporal slot: 1954 vs 1953"

6.2 — Retrieve per Sub-query

Mỗi sub-query retrieve independently: → Tập new documents `D_new` per hop → Extract claims từ `D_new` → Thêm vào evidence graph → Re-run GNN inference (không re-train) để update scores

6.3 — Update GNN Scores

Sau mỗi iteration:

1. `G_new = G U new_nodes U new_edges`
2. Re-run forward pass của GNN trên `G_new`
3. → update `claim_reliability_score`
4. → update `conflict_node_score`
5. → re-extract shortest paths

6.4 — Stopping Condition

Đưa vào LLM generation nếu rơi vào các trường hợp sau:

- (`claim_reliability_score` cao, `conflict_intensity_score` cao) HOẶC (`claim_reliability_score` thấp, `conflict_intensity_score` thấp)
- Nếu `conflict_intensity_score` < θ → break loop
- Nếu `iterations` ≥ `max_iter` → break, flag "unresolvable"
- Nếu GNN scores không thay đổi giữa iterations → break (new evidence không ảnh hưởng đến graph structure)